Facultad de Ingeniería

Programa de Ingeniería de Sistemas y Computación

Curso: Programación II

PARCIAL 2 - 15 de octubre del 2026

Nombre completo:	**C.C:**
Nombre completo:	**C.C:**

Resultados de aprendizaje a evaluar en el parcial:

R.A.1: Identifico los fundamentos del proceso de solución de problemas mediante la construcción de aplicaciones basadas en el uso de un computador, dentro del contexto de la programación orientada a objetos, y su aplicación en el contexto del profesional de Ingeniería de Sistemas y Computación.

R.A.2: Analizo un contexto de manera crítica, como punto de partida para la identificación de requisitos funcionales, pilar del conocimiento científico y génesis de la investigación formativa, aplicando principios éticos y morales para solucionar problemas del entorno, actuando de manera propositiva e interactuando local y globalmente a través de un uso efectivo de las tecnologías de la información.

R.A.3: Aplico los ejes conceptuales de la programación como parte de mi quehacer profesional, muestro capacidad en la toma de decisiones y pensamiento crítico con el fin de dar respuesta a las necesidades de los clientes mediante el diseño y la construcción de soluciones que mejoren la productividad y medien entre el conocimiento y los usuarios.

---

resuelva los siguientes puntos:

Contexto

La plataforma de comercio electrónico *ShopMaster* se prepara para la jornada de descuentos más grande del año, pero su arquitectura actual en Java es rígida y difícil de mantener. El líder técnico te ha entregado los siguientes **cinco requerimientos funcionales y de diseño** que debes solucionar en el motor backend (por consola), aplicando obligatoriamente los **5 patrones de diseño estructurales** vistos en clase (tú debes identificar qué patrón resuelve cada problemática y en qué parte del diseño aplicarlo):

1. **Gestión flexible de productos y paquetes promocionales:** En el catálogo de la tienda se venden artículos individuales (por ejemplo, un monitor de $800.000 o un teclado de $200.000), pero el área de mercadeo acaba de lanzar "Cajas Sorpresa" y "Combos Armados" (por ejemplo, un Combo Gamer que incluye un monitor, un teclado e incluso puede contener adentro otros sub-combos más pequeños, como un Kit de Limpieza con varios artículos). El sistema actual tiene código duplicado con muchos condicionales para saber si está cobrando un artículo suelto o una caja con más cosas adentro. Se necesita que el sistema trate exactamente igual a un producto individual que a un paquete compuesto, permitiendo calcular el precio total y mostrar el desglose completo de forma transparente sin importar cuántos niveles de paquetes haya uno dentro de otro.

2. **Servicios opcionales y acumulables sobre el pedido:** Una vez que se tiene listo el pedido base con su costo inicial, el usuario puede elegir agregarle diferentes beneficios opcionales de post-venta en tiempo de ejecución antes de pagar: una Póliza de Garantía Extendida (que incrementa el valor actual en un 10%), un Empaque de Regalo Premium (que suma $15.000 fijos) y un Manejo Prioritario Frágil (que suma $25.000 fijos). El cliente puede no elegir ninguno, elegir uno solo, combinarlos todos en cualquier orden o incluso aplicar doble empaque de regalo si así lo desea. Está prohibido usar herencia tradicional creando subclases para cada combinación posible (ej. PedidoConGarantiaYRegalo) ni llenar la clase principal del pedido con banderas booleanas (boolean tieneGarantia).

3. **Expansión a Estados Unidos e integración con pasarela de pagos externa:** Toda la plataforma ShopMaster fue programada originalmente para el mercado colombiano, realizando sus cobros a través de un contrato estándar interno llamado ProcesadorPago, el cual define el método cobrarEnPesos(String usuario, double montoCOP). Sin embargo, como la empresa acaba de abrir operaciones para vender en Estados Unidos, adquirió los derechos de uso de una pasarela de pagos norteamericana llamada StripeUSA (cuyo código fuente **no se puede modificar** porque viene empaquetado por un proveedor externo). Esta pasarela procesa las compras mediante un método totalmente distinto y en inglés: chargeInDollars(double amountUSD, String customerEmail, String merchantKey). Necesitas lograr que el sistema actual pueda conectarse y usar StripeUSA como si fuera un ProcesadorPago local más de la tienda, haciendo la conversión de moneda (1 USD = 4.000 COP) y enviando la clave de comercio requerida de manera transparente sin alterar el resto del código de la plataforma.

4. **Barrera de seguridad, control de cupos y auditoría:** Por normativas financieras, ninguna petición de cobro debe llegar directamente al procesador de pagos real ni a las "librerías bancarias" (tus clases simuladas de cobro) sin pasar antes por un control estricto. Se requiere un intermediario que se ponga exactamente en medio (fingiendo ser el procesador de pagos ante los ojos del sistema) e intercepte la solicitud:

   - Si el monto supera los $5.000.000 COP o si el nombre del usuario aparece reportado como "BLOQUEADO", debe rechazar la transacción inmediatamente sin siquiera instanciar ni tocar el procesador bancario real.
   - Si los datos son válidos, debe dejar pasar la petición hacia el procesador real y dejar impreso en consola un registro de auditoría con el mensaje [AUDITORÍA: Transacción aprobada para...].

5. **Simplificación de la compra rápida ("One-Click Checkout"):** Actualmente, para simular una compra completa desde la clase Main, el programador cliente tendría que conocer e instanciar manualmente las clases de los productos, armar la estructura de combos, envolver el pedido en cada servicio adicional, configurar la conexión con el banco internacional y pasarla por el control de seguridad. Se requiere un único punto de entrada simplificado para que, desde el Main, baste con llamar a un método de alto nivel (por ejemplo, ejecutarCompraRapida(...)) que coordine y oculte toda la complejidad de los subsistemas internos.

Escenarios de Prueba Obligatorios en el Main:

El programa no debe pedir datos por teclado (Scanner). En su lugar, dentro del método main debes dejar programados y ejecutándose en consola los siguientes **3 casos de prueba** llamando a tu punto de entrada simplificado:

- **Caso de Prueba 1 (Compra Exitosa Completa):**
  - Crea un combo principal (ej. "Combo Setup Gamer") que contenga al menos dos productos individuales (ej. Monitor de $800.000 y Teclado de $200.000) y además tenga adentro otro sub-combo más pequeño (ej. "Kit de Limpieza" con dos productos de $25.000 y $15.000).
  - Ejecuta la compra para un usuario válido (ej. "cliente_vip"), activando los extras de **Póliza de Garantía Extendida** y **Empaque de Regalo**. El sistema debe mostrar el árbol de productos, el cálculo con los recargos, la conversión a dólares en la pasarela de EE. UU. y el mensaje final de auditoría aprobada.

- **Caso de Prueba 2 (Rechazo por Tope de Dinero):**
  - Crea un producto individual costoso (ej. "Servidor Empresarial" de $6.000.000 COP) y agrégale el extra de **Manejo Prioritario Frágil**.
  - Intenta procesarlo con un usuario normal. La barrera de seguridad debe interceptar el cobro, detectar que supera el límite de $5.000.000 COP y mostrar el rechazo en consola sin tocar la pasarela de Estados Unidos.

- **Caso de Prueba 3 (Rechazo por Usuario Reportado):**
  - Usa un producto económico (ej. un Mouse de $150.000 COP), pero envía como nombre de cliente exactamente la palabra "BLOQUEADO".
  - La barrera de seguridad debe impedir el paso inmediatamente y mostrar la alerta en consola.

---

1. **(2.0 pts) Implementación en Java de los 5 patrones de diseño estructurales (Entrega de código):** desarrollo completo y funcional del proyecto en consola resolviendo los 5 requerimientos del contexto planteado, aplicando buenas prácticas de programación orientada a objetos (interfaces, composición, delegación y polimorfismo), respetando las reglas del dominio y ejecutando los tres escenarios de prueba solicitados en el método main.

2. **(3.0 pts) Sustentación, análisis arquitectónico y defensa del código (10 minutos):** demostración oral compartiendo pantalla en IntelliJ IDEA donde el estudiante identifica qué patrón estructural aplicó en cada uno de los cinco requerimientos.
